Introduction to Intensive Programming in C++

Lecture 7b : A few words about data structures

Elias TSIGARIDAS

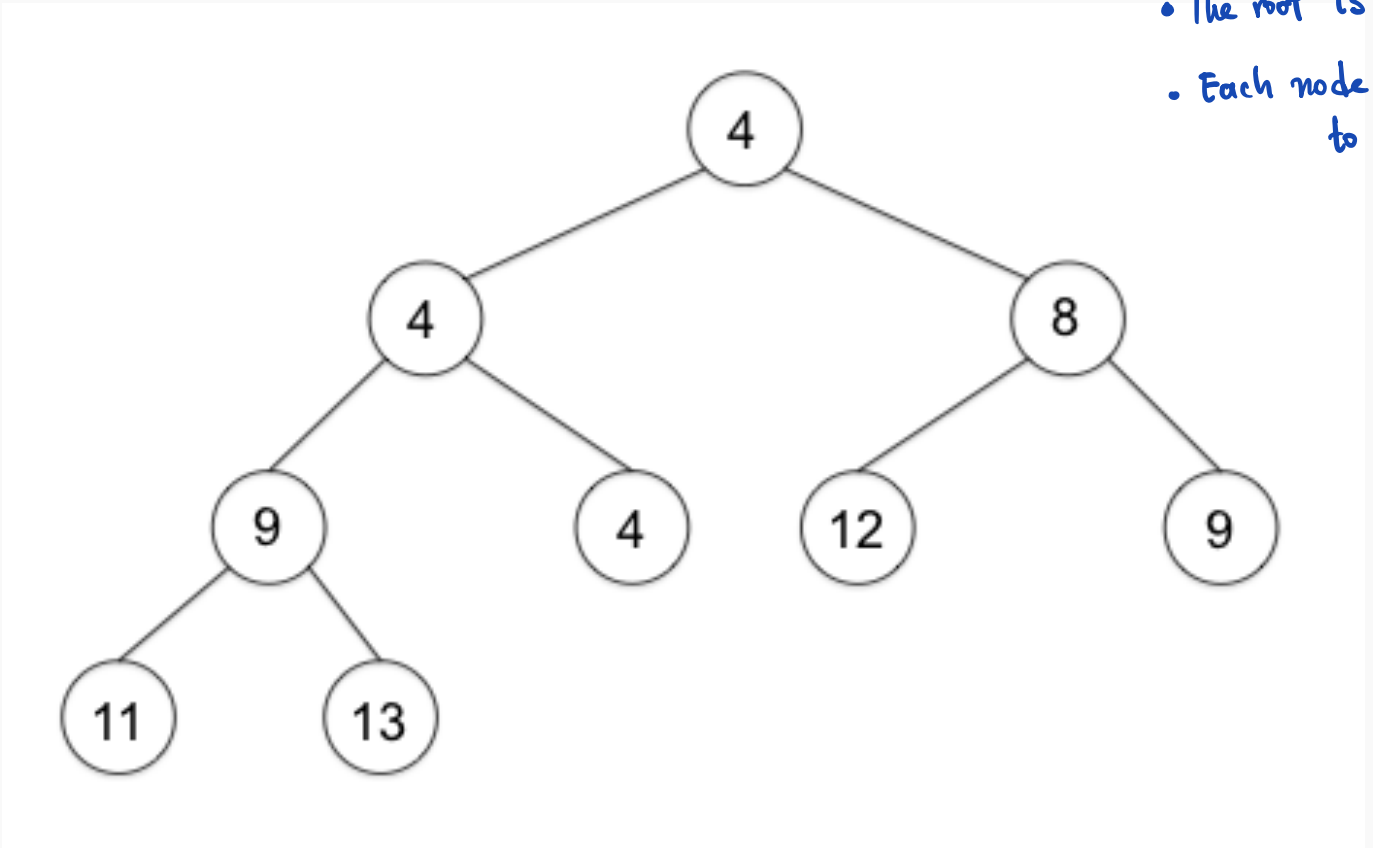
École Polytechnique

Contents

The Heap data structure (as a tree)

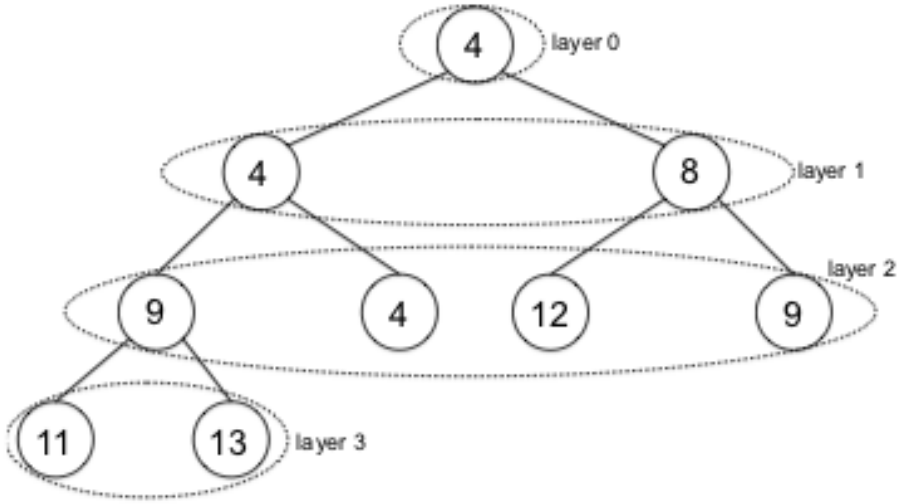
complete binary

- The root is the min of all keys
- Each node is less or equal to its children



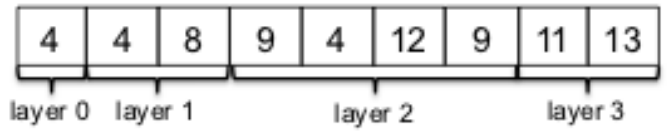
Operation	Running time
INSERT	$O(\log n)$
EXTRACTMIN	$O(\log n)$
FINDMIN	$O(1)$
HEAPIFY	$O(n)$
DELETE	$O(\log n)$

The Heap data structure (as a tree or an array)



(a) Tree representation

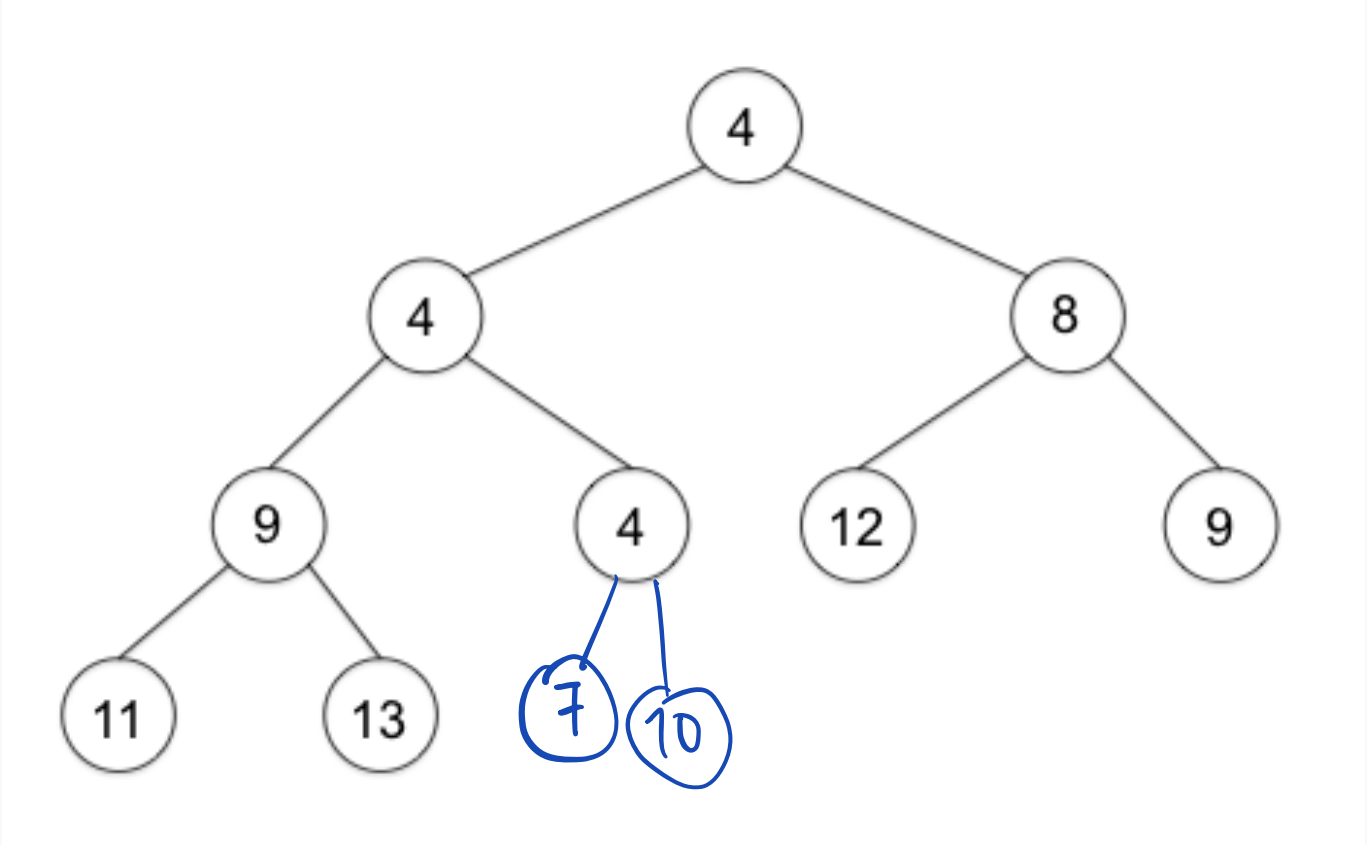
Position of parent	$\lfloor i/2 \rfloor$ (provided $i \geq 2$)
Position of left child	$2i$ (provided $2i \leq n$)
Position of right child	$2i+1$ (provided $2i+1 \leq n$)



(b) Array representation

The Heap data structure :: Insert

Insert : 7, 10, 5

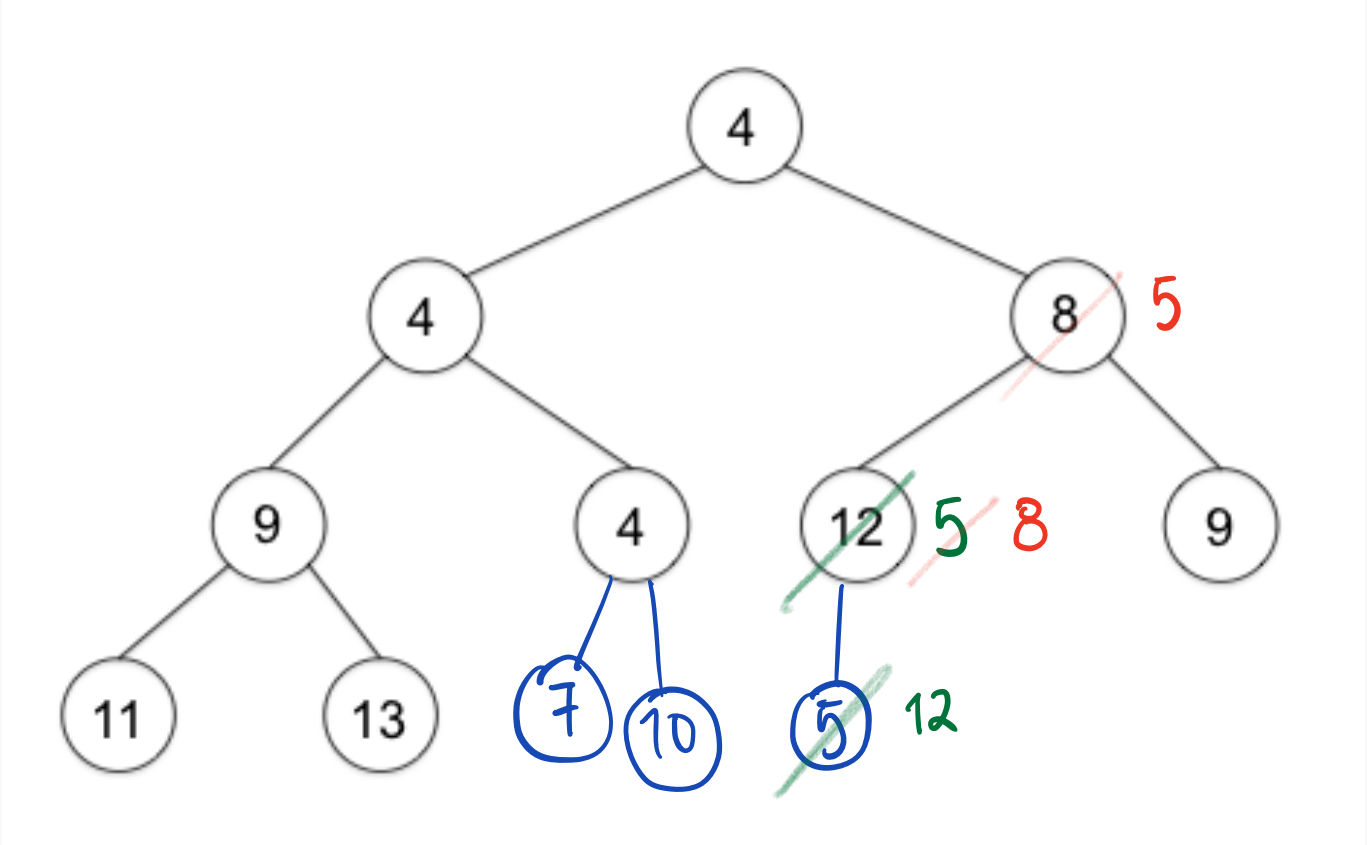


O(lgn)

The Heap data structure :: Insert

Insert : 7, 10, 5

- The root is the min of all keys
- Each node is less or equal to its children



O(lgn)

Contents

Union-Find

- We have n items
- Maintains a collection of disjoint sets
- Each of the n items is in exactly one set

- $items = \{1, 2, 3, 4, 5, 6\}$
- $collections = \{1, 4\}, \{3, 5, 6\}, \{2\}$
- $collections = \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$

- Supports two operations efficiently: `find(x)` and `union(x,y)`.

Union-Find

- $items = \{1, 2, 3, 4, 5, 6\}$
- $collections = \{1, 4\}, \{3, 5, 6\}, \{2\}$
- `find(x)` returns a representative item from the set that x is in
  - `find(1) = 1`
  - `find(4) = 1`
  - `find(3) = 5`
  - `find(5) = 5`
  - `find(6) = 5`
  - `find(2) = 2`
- a and b are in the same set if and only if `find(a) == find(b)`

Union-Find

- $\mathit{items} = \{1, 2, 3, 4, 5, 6\}$
- $\mathit{collections} = \{1, 4\}, \{3, 5, 6\}, \{2\}$

- `union(x, y)` merges the set containing x and the set containing y together.

  - `union(4, 2)`
  - $\mathit{collections} = \{1, 2, 4\}, \{3, 5, 6\}$
  - `union(3, 6)`
  - $\mathit{collections} = \{1, 2, 4\}, \{3, 5, 6\}$
  - `union(2, 6)`
  - $\mathit{collections} = \{1, 2, 3, 4, 5, 6\}$

Union-Find implementation

- Quick Union with path compression
- Extremely simple implementation
- Extremely efficient

```
struct union_find {
    vector<int> parent;
    union_find(int n) {
        parent = vector<int>(n);
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    // find and union
};
```

Union-Find implementation

```
// find and union

int find(int x) {
    if (parent[x] == x) {
        return x;
    } else {
        parent[x] = find(parent[x]);
        return parent[x];
    }
}

void unite(int x, int y) {
    parent[find(x)] = find(y);
}
```

Union-Find implementation (short)

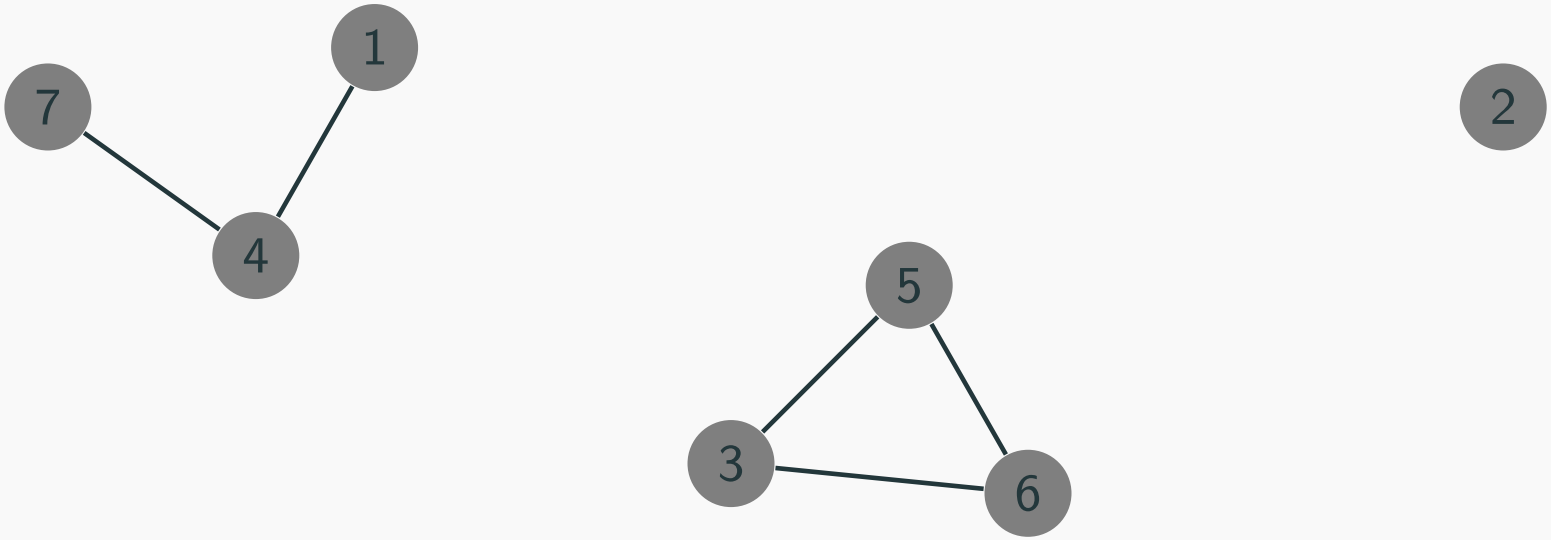
- If you're in a hurry...

```
#define MAXN 1000
int p[MAXN];

int find(int x) {
    return p[x] == x ? x : p[x] = find(p[x]); }
void unite(int x, int y) { p[find(x)] = find(y); }

for (int i = 0; i < MAXN; i++) p[i] = i;
```

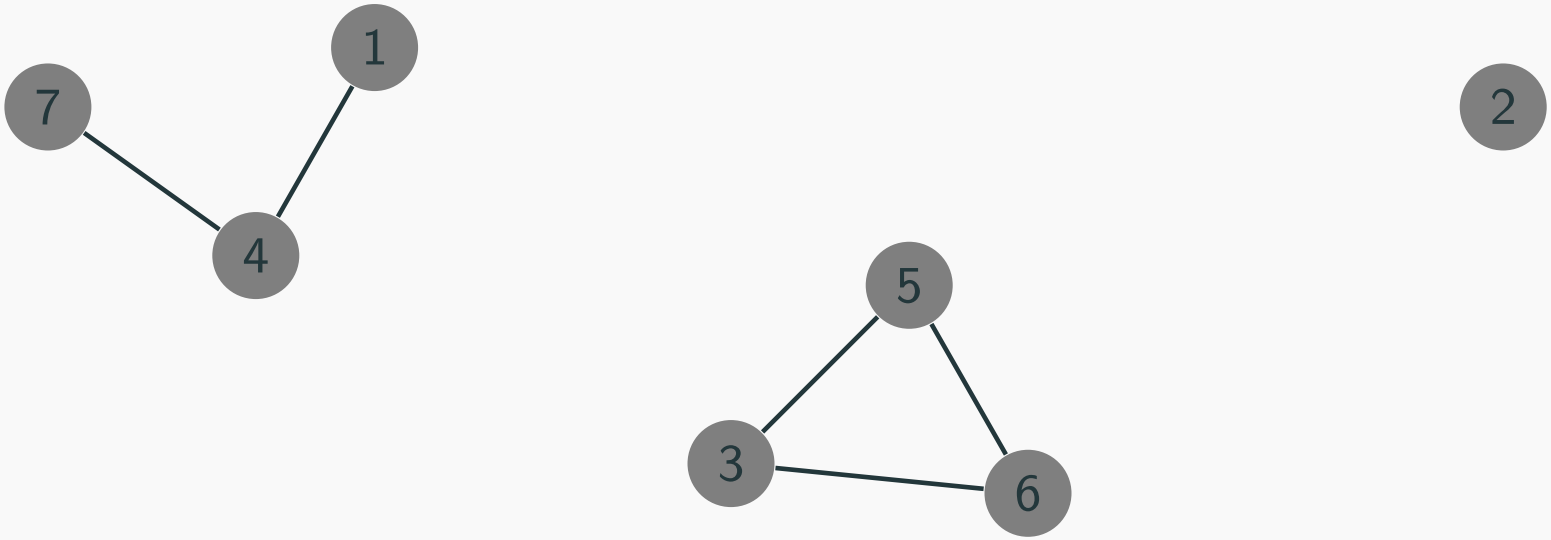
Union-Find applications

- Union-Find maintains a collection of disjoint sets
- When are we dealing with such collections?
- Most common example is in graphs

Disjoint sets in graphs

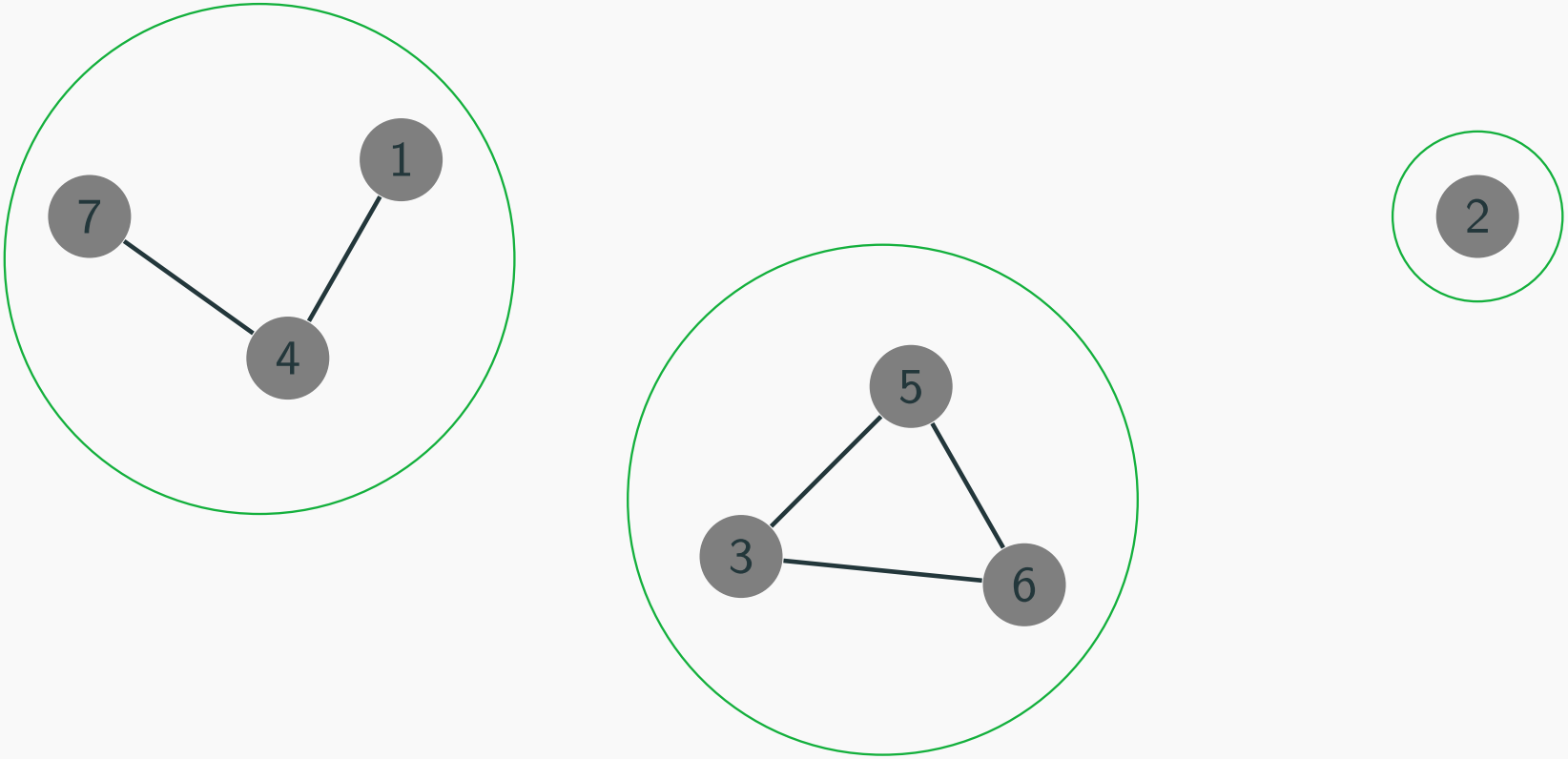


Disjoint sets in graphs



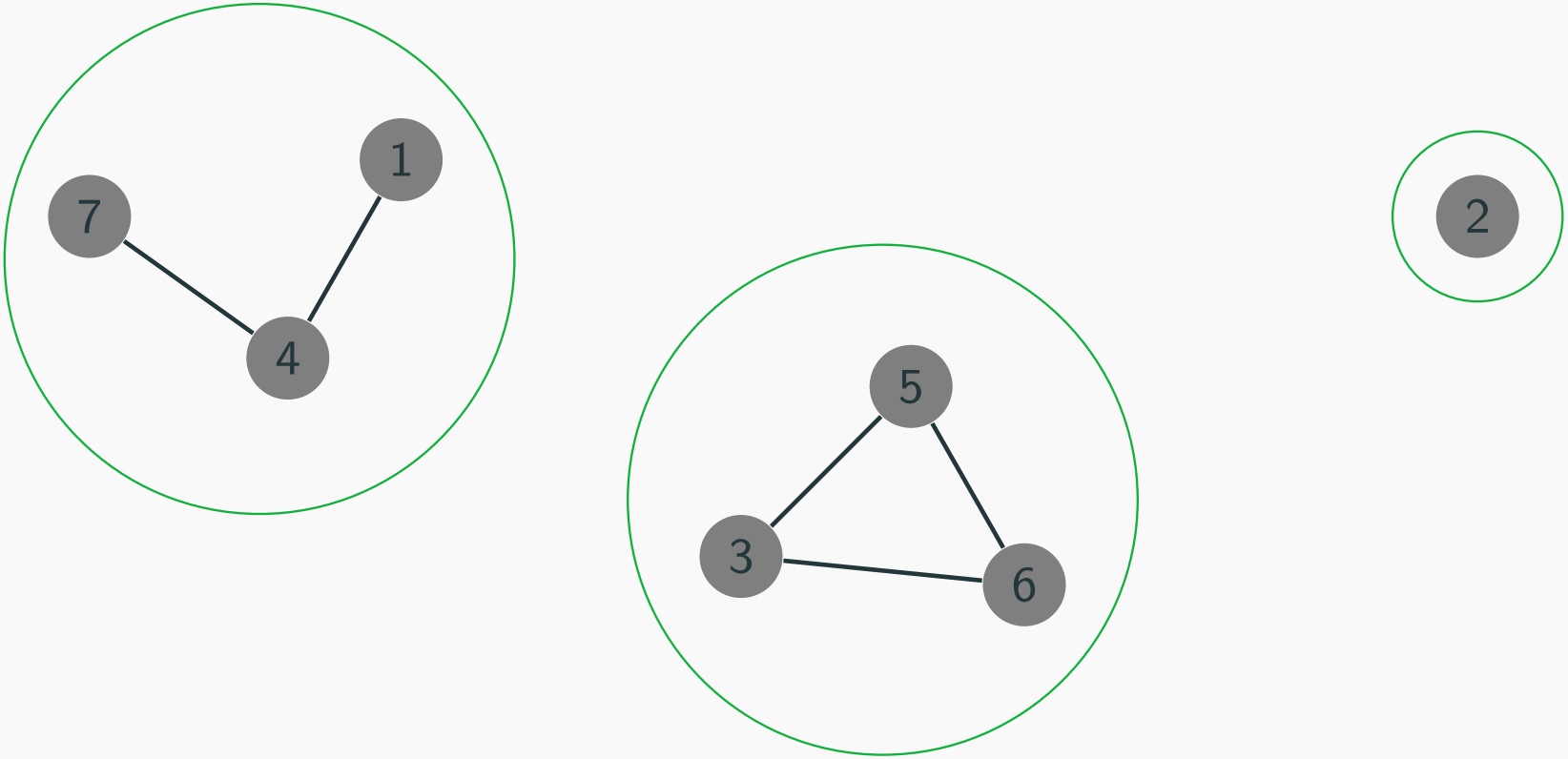
- $items = \{1, 2, 3, 4, 5, 6, 7\}$

Disjoint sets in graphs



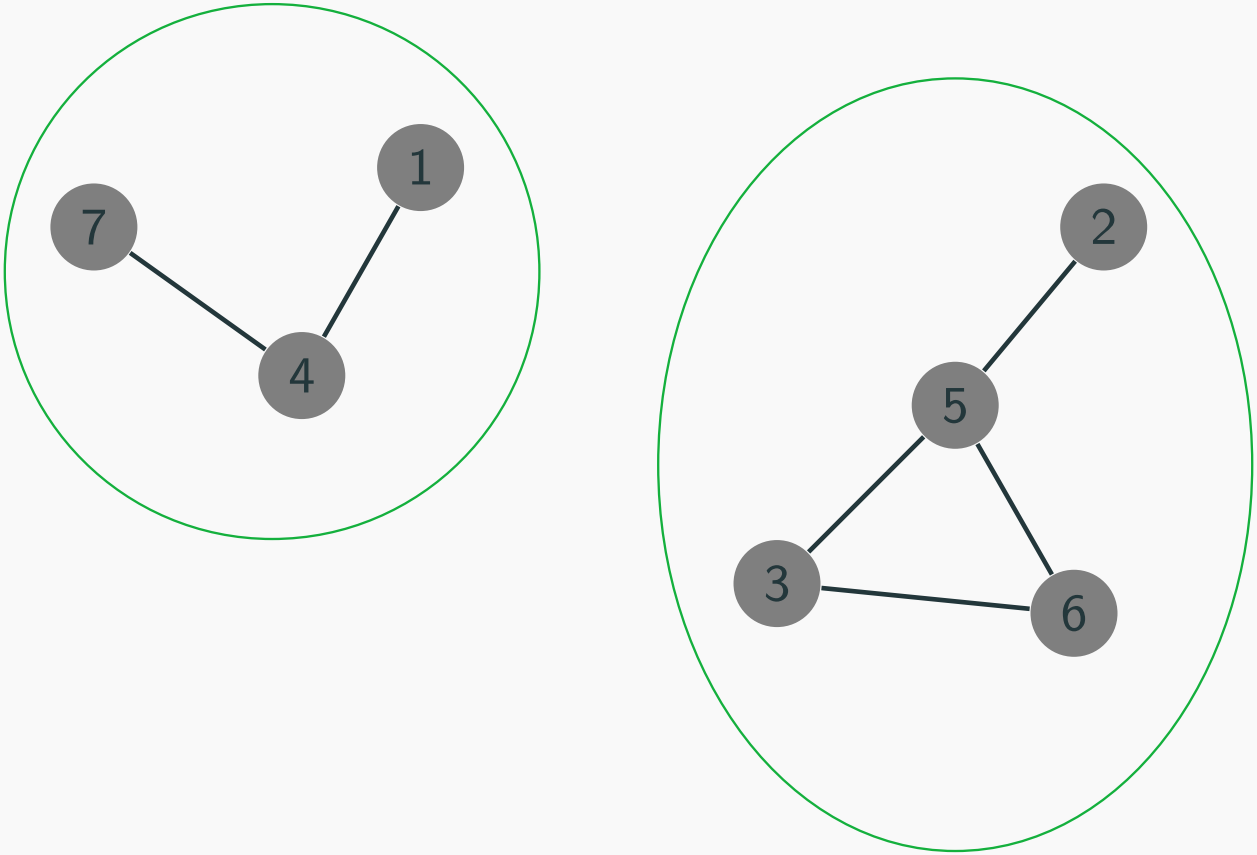
- $items = \{1, 2, 3, 4, 5, 6, 7\}$
- $collections = \{1, 4, 7\}, \{2\}, \{3, 5, 6\}$

Disjoint sets in graphs



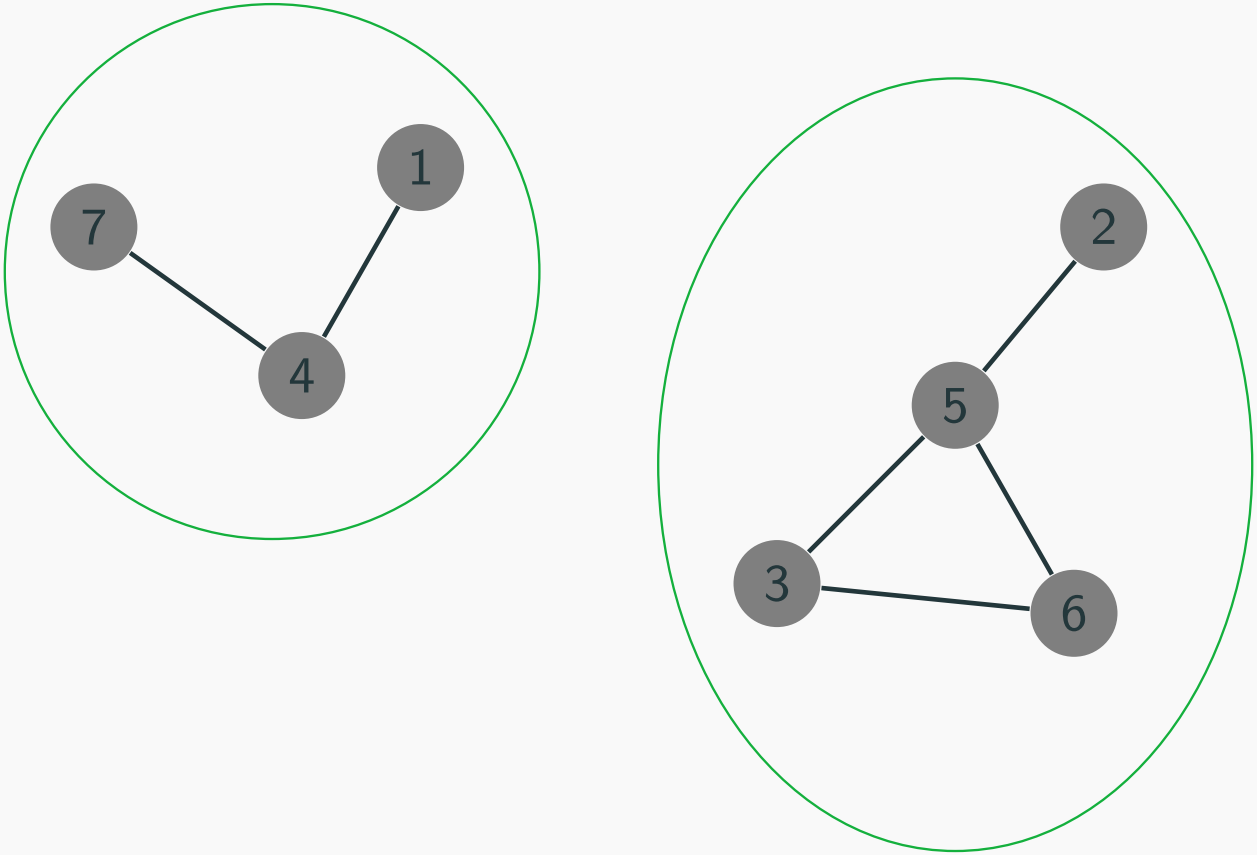
- $\mathit{items} = \{1, 2, 3, 4, 5, 6, 7\}$
- $\mathit{collections} = \{1, 4, 7\}, \{2\}, \{3, 5, 6\}$
- `union(2, 5)`

Disjoint sets in graphs



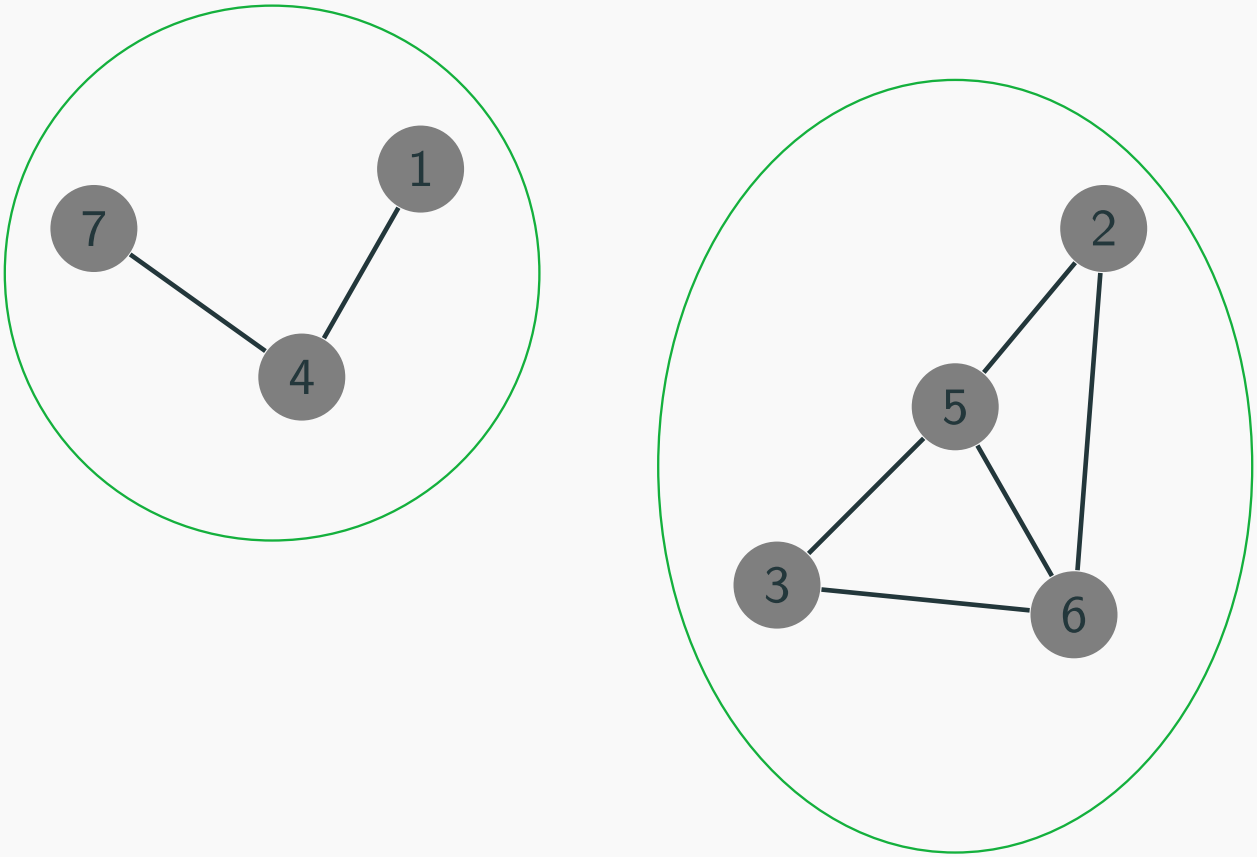
- *items* $= \{1, 2, 3, 4, 5, 6, 7\}$
- *collections* $= \{1, 4, 7\}, \{2, 3, 5, 6\}$

Disjoint sets in graphs



- $\mathit{items} = \{1, 2, 3, 4, 5, 6, 7\}$
- $\mathit{collections} = \{1, 4, 7\}, \{2, 3, 5, 6\}$
- `union(6, 2)`

Disjoint sets in graphs



- *items* $= \{1, 2, 3, 4, 5, 6, 7\}$
- *collections* $= \{1, 4, 7\}, \{2, 3, 5, 6\}$